

Линейная модель, функция потерь и оптимизация

Линейная регрессия как прозрачный пример того, что происходит внутри `fit`



Задача регрессии и линейное семейство

$$x \in \mathcal{X}$$

↓

$$f_{\beta}(x) \in \mathbb{R}$$

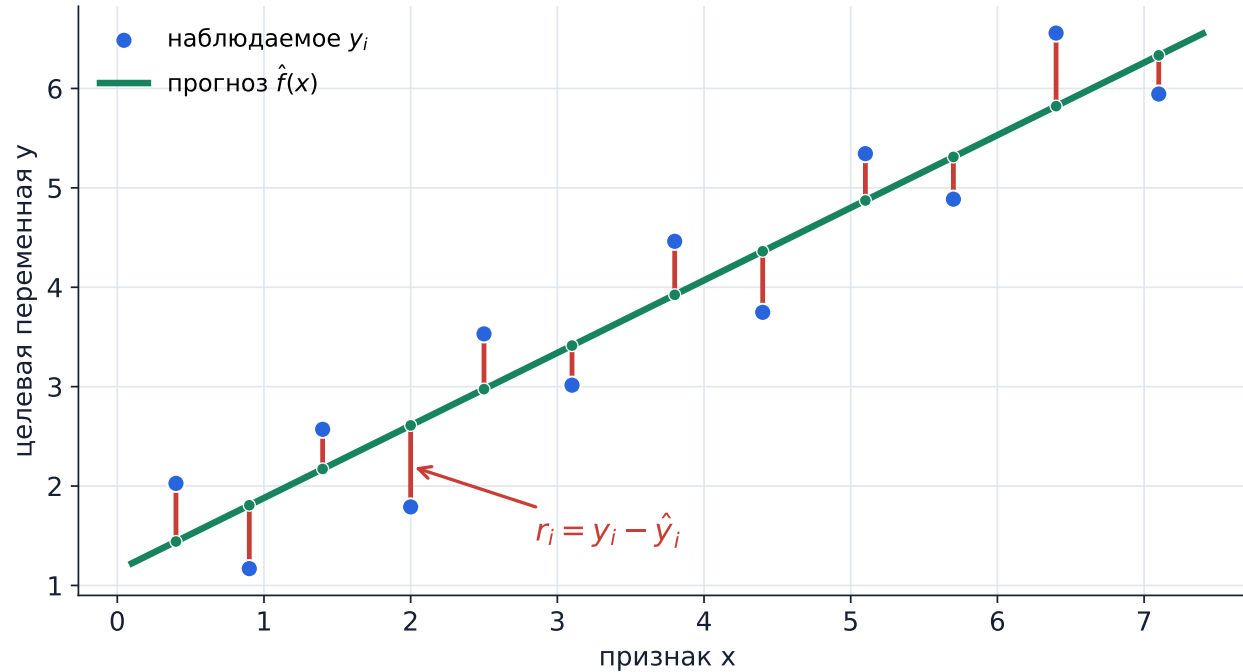
числовой прогноз

$$f_{\beta}(x) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

Разные значения β задают разные функции одного семейства.

Обучение должно выбрать одну функцию из этого семейства.

После обучения: прогнозы и остатки



$$\hat{\beta} = (\hat{\beta}_0, \dots, \hat{\beta}_p)^\top$$

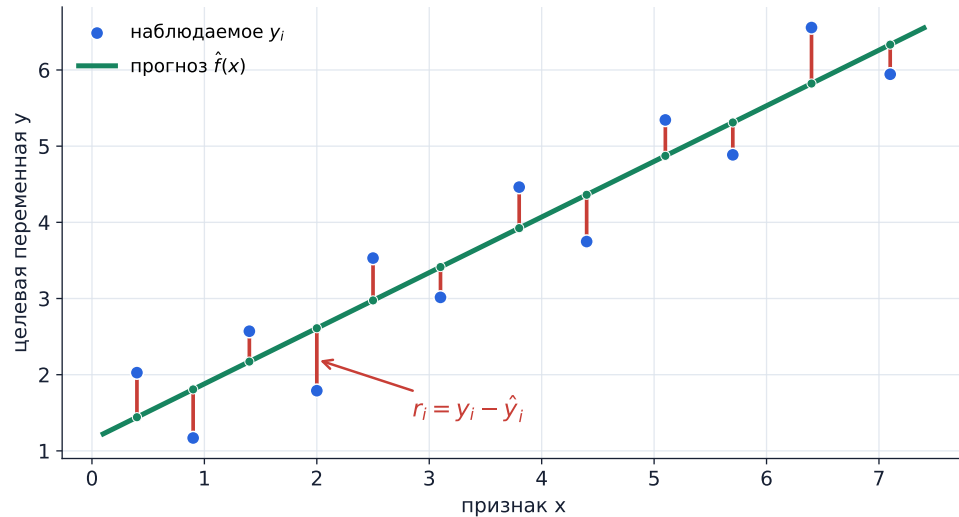
$$\hat{y}_i = f_{\hat{\beta}}(x_i)$$

$$r_i = y_i - \hat{y}_i$$

$r_i > 0$: модель недооценила наблюдаемое значение.

$r_i < 0$: модель переоценила его.

Метод наименьших квадратов



$$r_i(\beta) = y_i - f_\beta(x_i)$$

$$RSS(\beta) = \sum_{i=1}^n r_i(\beta)^2$$

$$\hat{\beta} \in \arg \min_{\beta} RSS(\beta)$$

Модель задаёт семейство прогнозов; RSS задаёт правило выбора параметров внутри него.

Аналитическое решение: короткое напоминание

$$\frac{\partial RSS}{\partial \beta_0} = 0, \quad \frac{\partial RSS}{\partial \beta_1} = 0$$



$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

Решение существует в таком виде, если признак имеет ненулевую вариацию.

Матричная запись для нескольких признаков

$$X \in \mathbb{R}^{n \times q}, \quad \beta \in \mathbb{R}^q, \quad y \in \mathbb{R}^n$$

$$\hat{y} = X\beta, \quad r = y - X\beta$$

$$\hat{\beta} \in \arg \min_{\beta} \|y - X\beta\|_2^2$$

строки X

обучающие объекты

столбцы X

признаки и свободный член

$x\beta$

по одному прогнозу на объект

Нормальные уравнения — условие оптимума

$$X^{\top} X \hat{\beta} = X^{\top} y$$

если столбцы X линейно независимы

$$\hat{\beta} = (X^{\top} X)^{-1} X^{\top} y$$

На практике least-squares-задачу решают устойчивыми численными методами, не вычисляя обратную матрицу явно.

От формулы к вычислению

NUMPY: ЧИСЛЕННЫЙ РЕШАТЕЛЬ

```
beta_hat, *_ = np.linalg.lstsq(  
    X, y, rcond=None  
)
```

Решается исходная задача
 $\min_{\beta} \|y - X\beta\|_2^2$.

SCIKIT-LEARN: ESTIMATOR

```
model = LinearRegression()  
model.fit(X_train, y_train)  
y_pred = model.predict(X_test)
```

Практические методы используют
QR/SVD, а не явное обращение $X^T X$.

При нескольких решениях `lstsq`
выбирает решение минимальной
евклидовой нормы.

Точная мультиколлинеарность

$$x_2 = 2x_1$$

$$\begin{aligned}\beta'_1 x_1 + \beta'_2 x_2 &= (\beta_1 - 2t)x_1 + (\beta_2 + t)2x_1 \\ &= \beta_1 x_1 + \beta_2 x_2.\end{aligned}$$

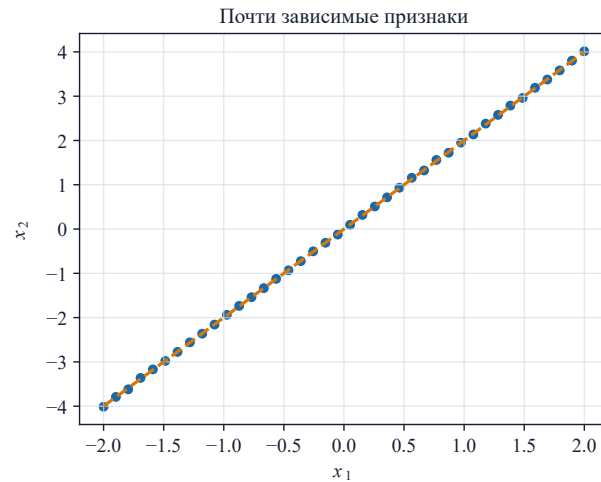
$$\beta'_1 = \beta_1 - 2t, \quad \beta'_2 = \beta_2 + t$$

одинаковые прогнозы $\nRightarrow$ одинаковые коэффициенты

`lstsq` возвращает конкретное решение, но не добавляет информации, которой нет в данных.

Приближённая мультиколлинеарность

$$x_2 \approx 2x_1$$



Малое возмущение ответов

	$\hat{\beta}_1$	$\hat{\beta}_2$	RMSE(Δy)
данные A	-3.48	3.23	0
малое возмущение	-4.11	3.54	0.013

$\|\Delta \hat{\beta}\|_2 = 0.70$

Небольшое изменение данных может заметно изменить отдельные коэффициенты.

Прогнозы на знакомых комбинациях признаков могут измениться значительно слабее.

Для прогноза и для интерпретации коэффициентов важны разные свойства решения.

Карта метрик регрессии

MSE

средний квадрат
ошибки

усиливает крупные
промахи

RMSE

корень из MSE

единицы целевой
переменной

MAE

средний модуль
ошибки

линейная цена
величины промаха

R^2

сравнение с
постоянным
прогнозом

безразмерная
величина

MAPE

средняя
относительная
ошибка

проценты от
фактического
значения

Метрика — это способ агрегировать ошибки уже полученных прогнозов.

MSE, RMSE и MAE: крупные ошибки

$$MSE = \frac{1}{m} \sum_i (y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{\frac{1}{m} \sum_i (y_i - \hat{y}_i)^2}$$

$$MAE = \frac{1}{m} \sum_i |y_i - \hat{y}_i|$$

$$A = (2, 2, 2, 2)$$

$$MAE_A = 2, \quad RMSE_A = 2$$

$$B = (0, 0, 0, 8)$$

$$MAE_B = 2, \quad RMSE_B = 4$$

MAE одинаково оценивает общий модуль ошибок; RMSE сильнее реагирует на один крупный промах.

Коэффициент детерминации R^2

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

$\bar{y}$ — среднее фактических значений на оцениваемой выборке

$$R^2 = 1$$

идеальные прогнозы

$$R^2 = 0$$

уровень постоянного прогноза

$$\hat{y}_i = \bar{y}$$

$$R^2 < 0$$

квадратичная ошибка выше этой
точки отсчёта

На test это метрика готовых прогнозов; привычную фразу «доля объяснённой вариации» безопасно применять к OLS с константой на той же обучающей выборке.

MAPE: относительная ошибка

$$MAPE = \frac{1}{m} \sum_i \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

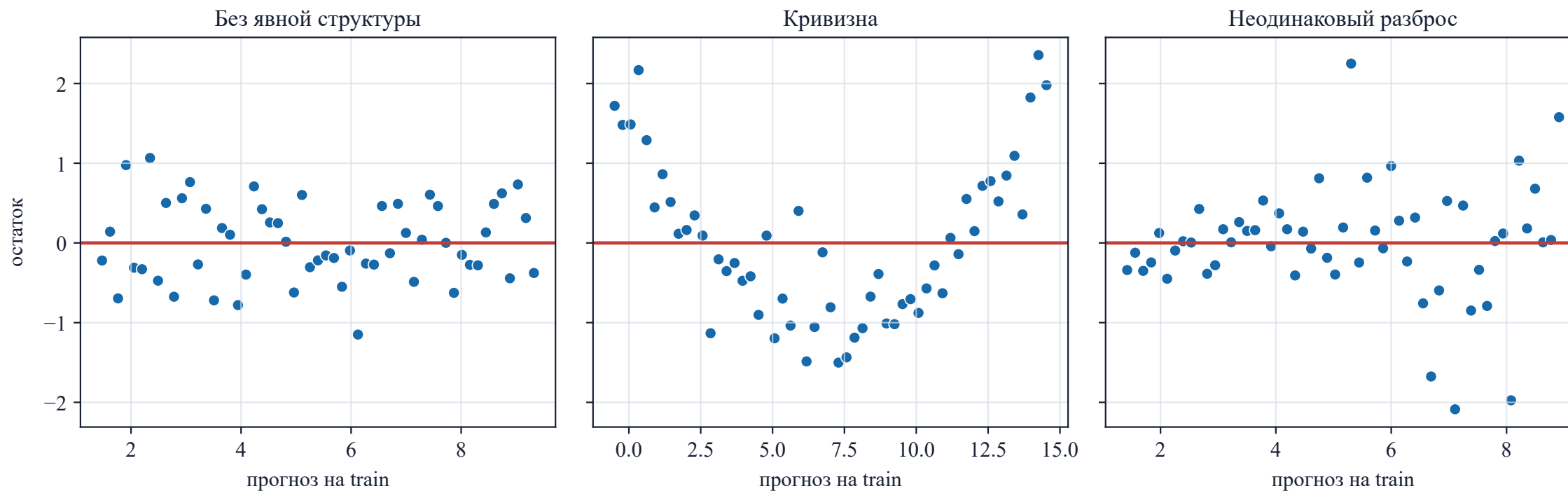
После умножения на 100% результат читают в процентах.

Удобна, когда важен размер ошибки относительно масштаба фактического значения.

$y_i = 0$: выражение не определено.

При $|y_i| \approx 0$ небольшая абсолютная ошибка может дать огромное значение.

Остатки: есть ли структура ошибки?



Явной структуры не видно.

Кривизна: средняя зависимость описана не полностью.

Разброс ошибки зависит от области данных.

Метрика показывает общий размер ошибки; график остатков помогает увидеть, где и как модель ошибается.

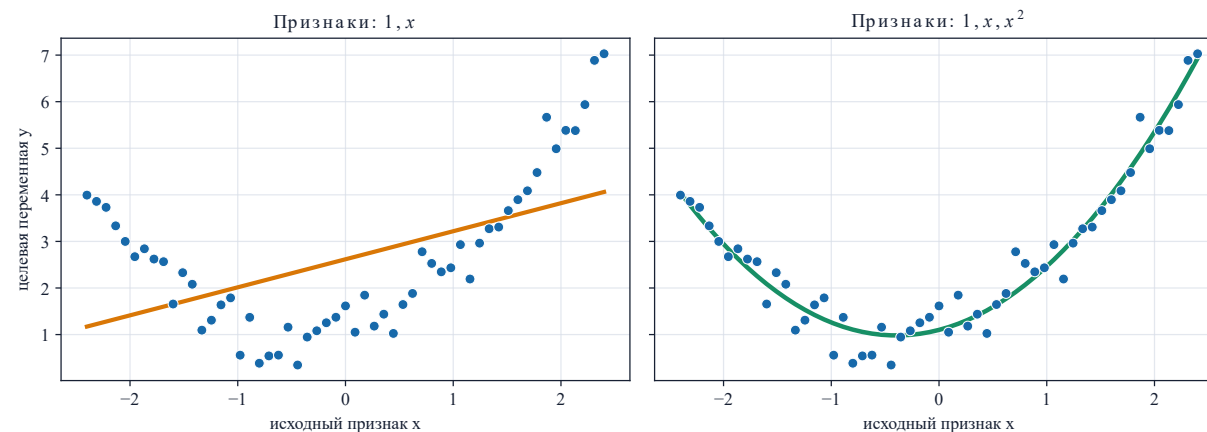
Кривизна в остатках → изменить семейство моделей

$$f_1(x) = \beta_0 + \beta_1 x$$

↓

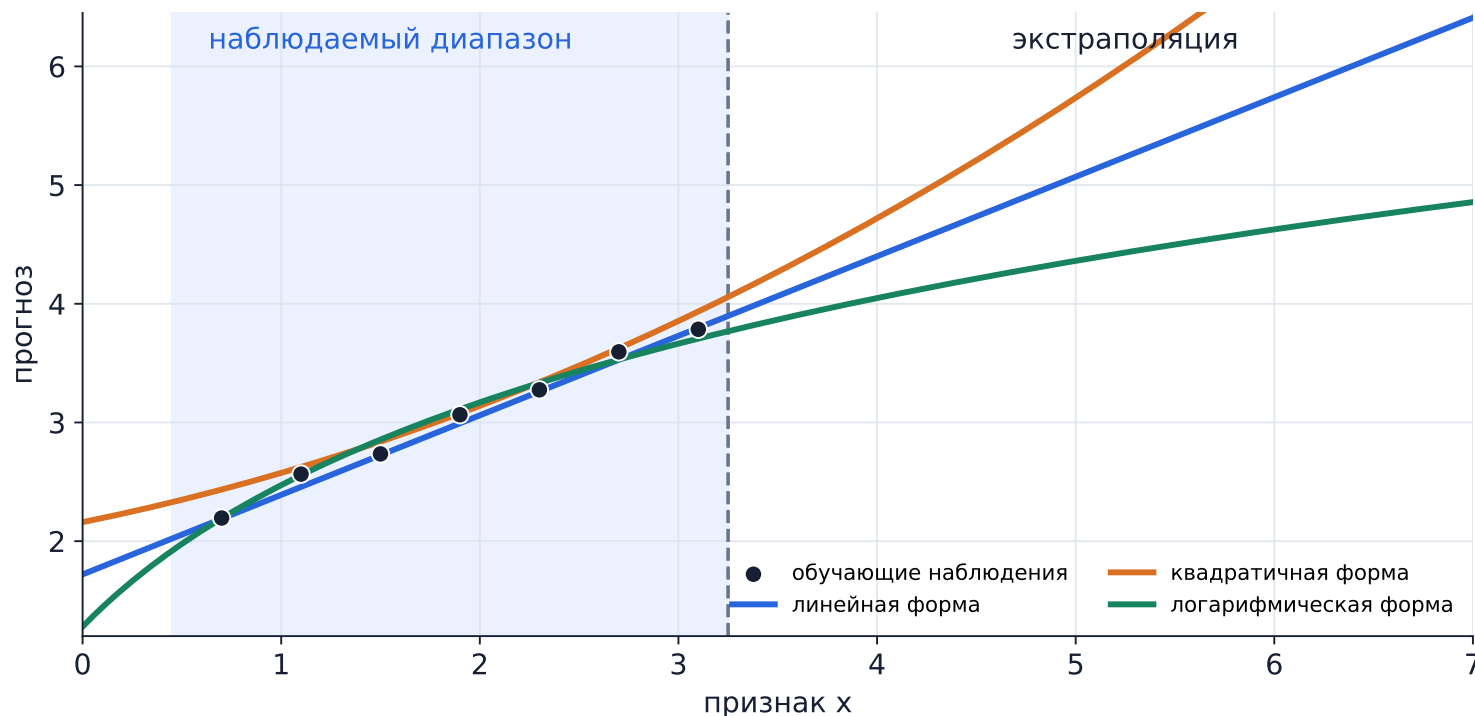
$$f_2(x) = \beta_0 + \beta_1 x + \beta_2 x^2$$

Модель нелинейна по исходному x , но остаётся линейной по коэффициентам.



После изменения признаков переобучаем модель и повторяем ту же диагностику.

Экстраполяция: продолжения расходятся



Внутри наблюдаемого диапазона несколько функциональных форм могут давать близкие прогнозы.

За пределами данных поведение определяется выбранной формой модели.

Хорошее качество внутри диапазона не обосновывает форму далёкой экстраполяции.

Таким образом

Модель: столбцы X задают семейство прогнозов $X\beta$.

Обучение: МНК выбирает параметры, минимизирующие сумму квадратов остатков.

Вычисление: `lstsq` решает least-squares-задачу численно; нормальные уравнения характеризуют оптимум.

Мультиколлинеарность: коэффициенты могут быть неединственными или нестабильными, даже если прогнозы меняются слабее.

Проверка: метрики показывают общий размер ошибки, остатки — её структуру, а форма модели определяет экстраполяцию.

Четыре роли, которые важно различать

Модель

Какие прогнозы
можно представить?

Функция потерь и целевая функция

Какие ошибки
дороги при
обучении?

Оптимизатор или решатель

Как найти
параметры?

Метрика

Как оценить уже
обученную модель?

модель + функция потерь + решатель → обученные
параметры

обученная модель + данные для оценки → метрики и
диагностика

Остаток превращается в цену ошибки

ОШИБКА СО ЗНАКОМ

$$r_i = y_i - \hat{y}_i$$

→

ЦЕНА ОШИБКИ

$$L(y_i, \hat{y}_i)$$

$$L_{\text{sq}}(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2 = r_i^2$$

Функция потерь задаёт правило, по которому ошибка отдельного прогноза превращается в численную величину.

$$r_i = -2 \Rightarrow L_{\text{sq}} = 4$$

$$r = 10 \Rightarrow L_{\text{sq}} = 100, \quad r = 20 \Rightarrow L_{\text{sq}} = 400 = 4 \cdot 100$$

От функции потерь к эмпирическому риску



Сначала оцениваем ошибку каждого прогноза, затем агрегируем значения по обучающей выборке.

$$L_i(\theta) = L(y_i, f_\theta(x_i))$$

$$R_n(\theta) = \frac{1}{n} \sum_{i=1}^n L_i(\theta)$$

$$\hat{\theta} \in \arg \min_{\theta} R_n(\theta)$$

Функция потерь относится к одному объекту; эмпирический риск задаёт цель обучения по всей выборке.

Целевая функция и метрика: одна формула, разные роли

ОБУЧЕНИЕ

$$R_{\text{train}}(\theta) = \frac{1}{n_{\text{train}}} \sum_{i \in \text{train}} L(y_i, f_{\theta}(x_i))$$

θ меняется: целевая функция выбирает параметры.

ОЦЕНИВАНИЕ

$$M_{\text{eval}} = M(y, \hat{y}(\hat{\theta}))$$

Параметры $\hat{\theta}$ уже зафиксированы: метрика оценивает результат.

MSE может быть целевой функцией на train и метрикой на validation/test

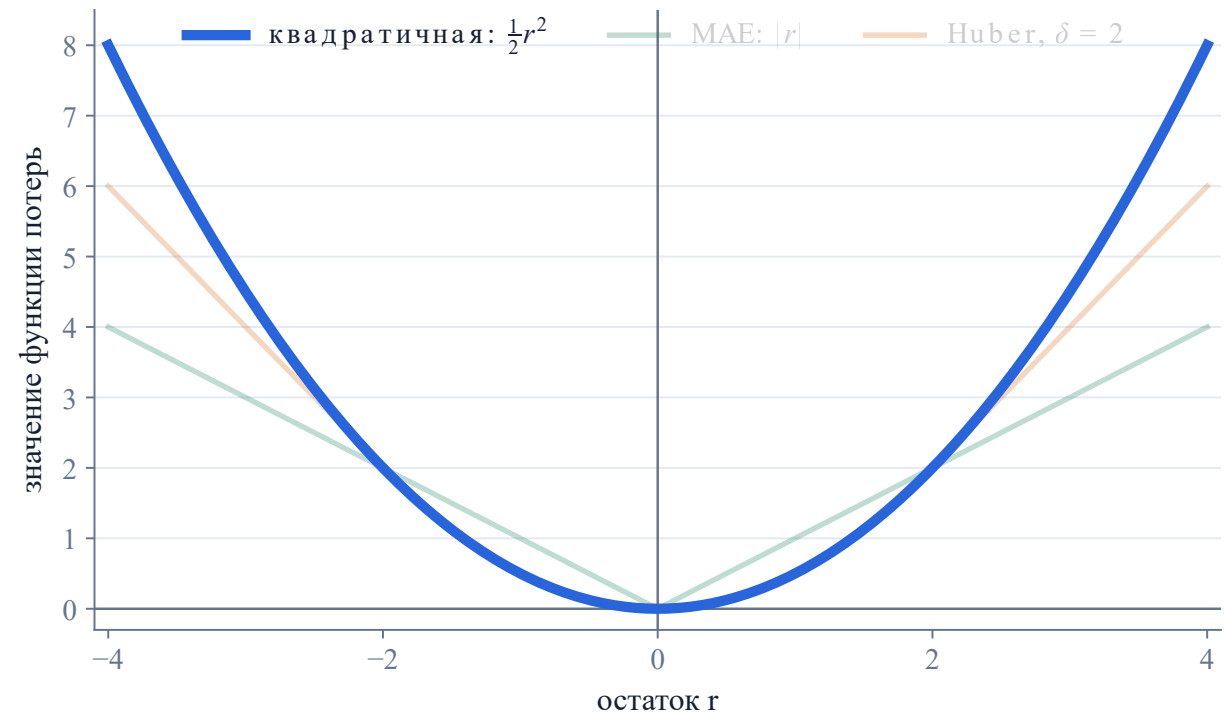
MSE как функция потерь

$$L_{sq}(r) = r^2$$

$$R_{MSE}(\theta) = \frac{1}{n} \sum_i (y_i - f_{\theta}(x_i))^2$$

$$\frac{d}{dr} r^2 = 2r$$

Чем больше $|r|$, тем быстрее растут loss и модуль производной.



На графике показана $\frac{1}{2}r^2$: положительный множитель не меняет минимум.

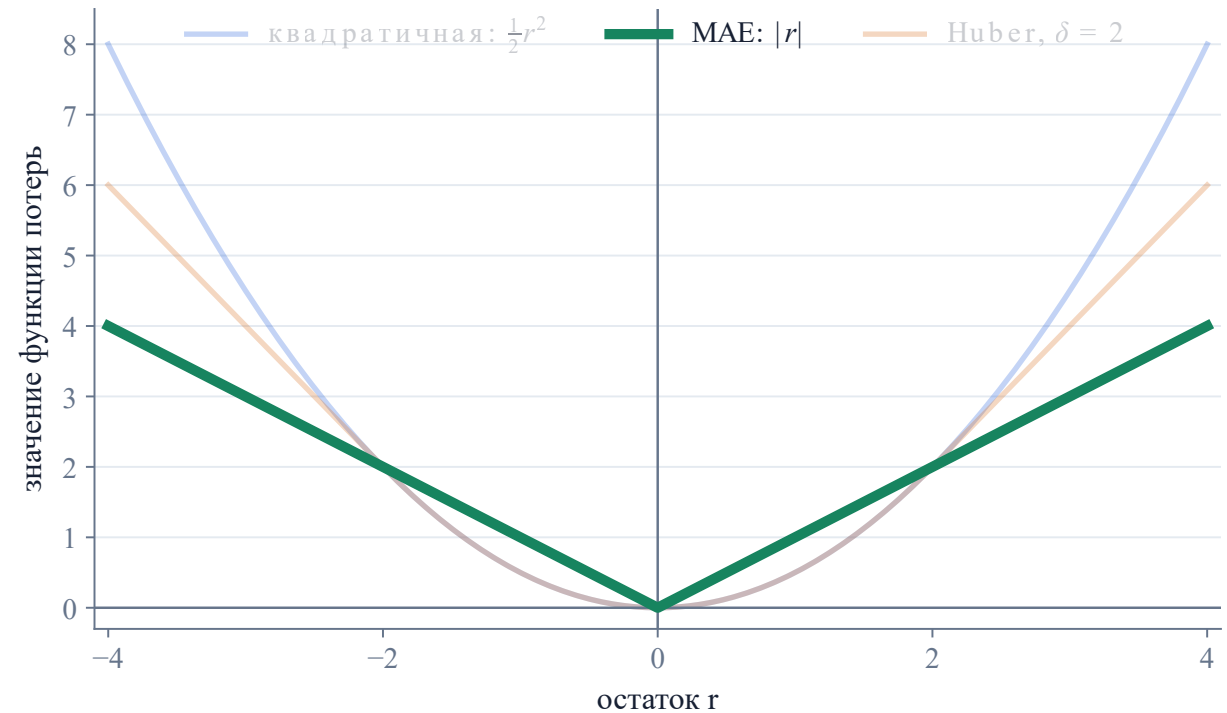
MAE как функция потерь

$$L_{\text{abs}}(r) = |r|$$

$$R_{\text{MAE}}(\theta) = \frac{1}{n} \sum_i |y_i - f_{\theta}(x_i)|$$

Цена ошибки растёт линейно по модулю остатка.

В точке $r = 0$ обычной производной нет.



Негладкость меняет требования к методу оптимизации, но не делает задачу некорректной.

Функция потерь определяет, что считать лучшим

КВАДРАТИЧНАЯ ФУНКЦИЯ

$$\arg \min_c \sum_i (y_i - c)^2 = \bar{y}$$

Лучший постоянный прогноз — среднее.

АБСОЛЮТНАЯ ФУНКЦИЯ

$$\hat{c}_{\text{MAE}} \in \arg \min_c \sum_i |y_i - c|$$

$$\hat{c}_{\text{MAE}} = \text{median}(y_1, \dots, y_n)$$

Лучший постоянный прогноз — медиана.

Функция потерь определяет, какой прогноз будет выбран при обучении.

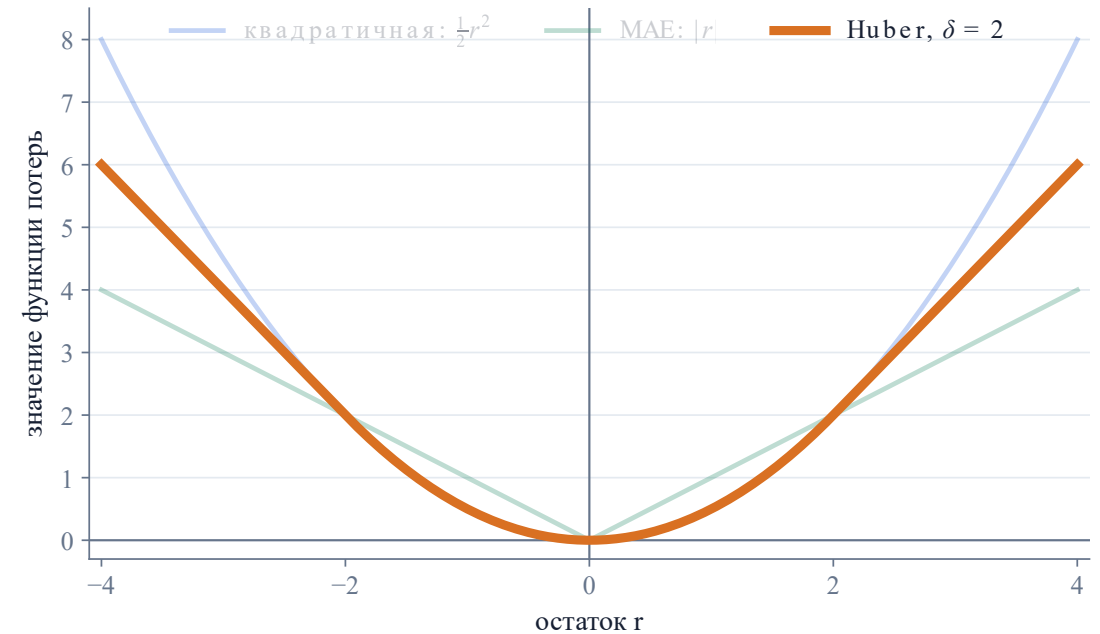
Huber: квадратичный центр, линейные хвосты

$$L_{\delta}(r) = \begin{cases} \frac{1}{2}r^2, & |r| \leq \delta, \\ \delta \left(|r| - \frac{1}{2}\delta \right), & |r| > \delta. \end{cases}$$

около нуля — гладкий
квадратичный режим

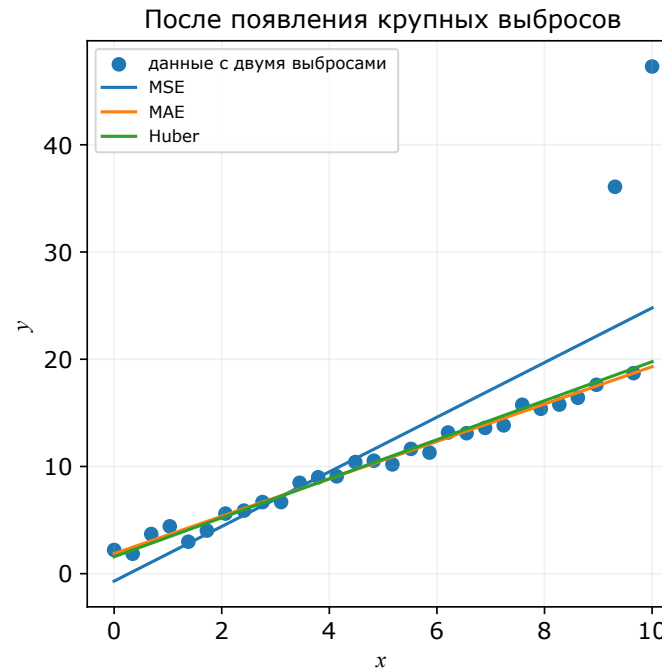
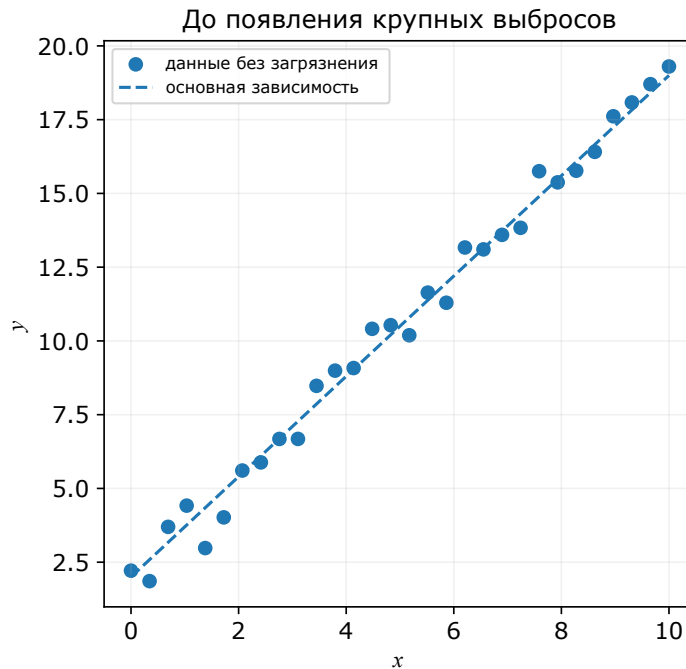
в хвостах — линейный рост

После порога модуль наклона перестаёт увеличиваться.



Одна модель — разные целевые функции

Крупные остатки по-разному меняют обученную линейную модель



$$\hat{y} = f_{\theta}(x)$$

Семейство допустимых функций остаётся тем же.

Правило выбора параметров меняется.

Поэтому обычно меняются и найденные коэффициенты.

одна модель + разные функции потерь $\implies$ разные параметры

Квантильная функция: цена ошибки зависит от направления

$$r = y - \hat{y}$$

$$\rho_{\tau}(r) = \tau \max(r, 0) + (1 - \tau) \max(-r, 0)$$

$r > 0$: недопрогноз

$r < 0$: перепрогноз

Дефицит в четыре раза дороже
излишка

$$\frac{\tau}{1 - \tau} = 4$$

$$\tau = 0.8$$

Асимметричная функция потерь кодирует разную цену недопрогноза и перепрогноза.

Функция потерь и целевая функция: итог

Модель

задаёт семейство допустимых прогнозов.

Эмпирический риск

усредняет потери и становится целевой функцией обучения.

Функция потерь

назначает цену ошибке одного объекта.

Выбор функции

меняет предпочтительное решение: среднее, медиану, компромисс Huber или асимметричный прогноз.

Запись $\arg \min$ определяет цель; следующий вопрос — как найти её минимум.

`arg min` задаёт цель, но не алгоритм

$$\hat{\theta} \in \arg \min_{\theta} R_n(\theta)$$

Запись говорит, **что** ищем: параметры с минимальным значением целевой функции.

Она не говорит, **как** эти параметры вычислить.

Для OLS есть специализированный решатель; для произвольной дифференцируемой целевой функции нужен более общий способ поиска.

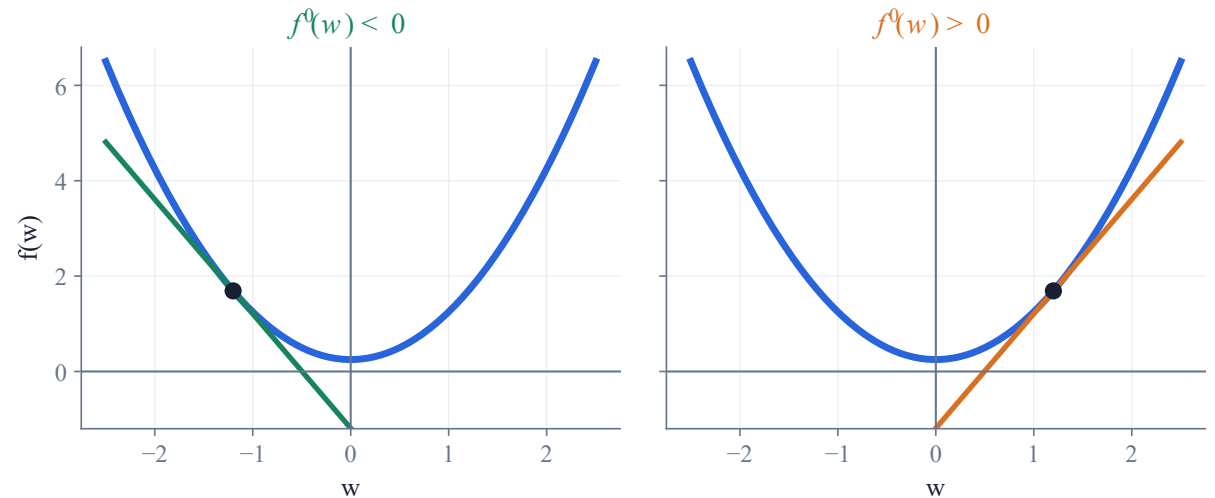
Производная: локальный наклон

$$f'(w) = \lim_{h \rightarrow 0} \frac{f(w + h) - f(w)}{h}$$

$$f(w) = (w - 3)^2, \quad f'(w) = 2(w - 3)$$

$f'(w) > 0$: движение вправо увеличивает функцию.

$f'(w) < 0$: движение вправо уменьшает функцию.



При нескольких параметрах нужен градиент

$$\nabla f(\theta) = \begin{pmatrix} \partial f / \partial \theta_1 \\ \vdots \\ \partial f / \partial \theta_d \end{pmatrix}$$

Каждая координата — локальное изменение функции при изменении одного параметра.

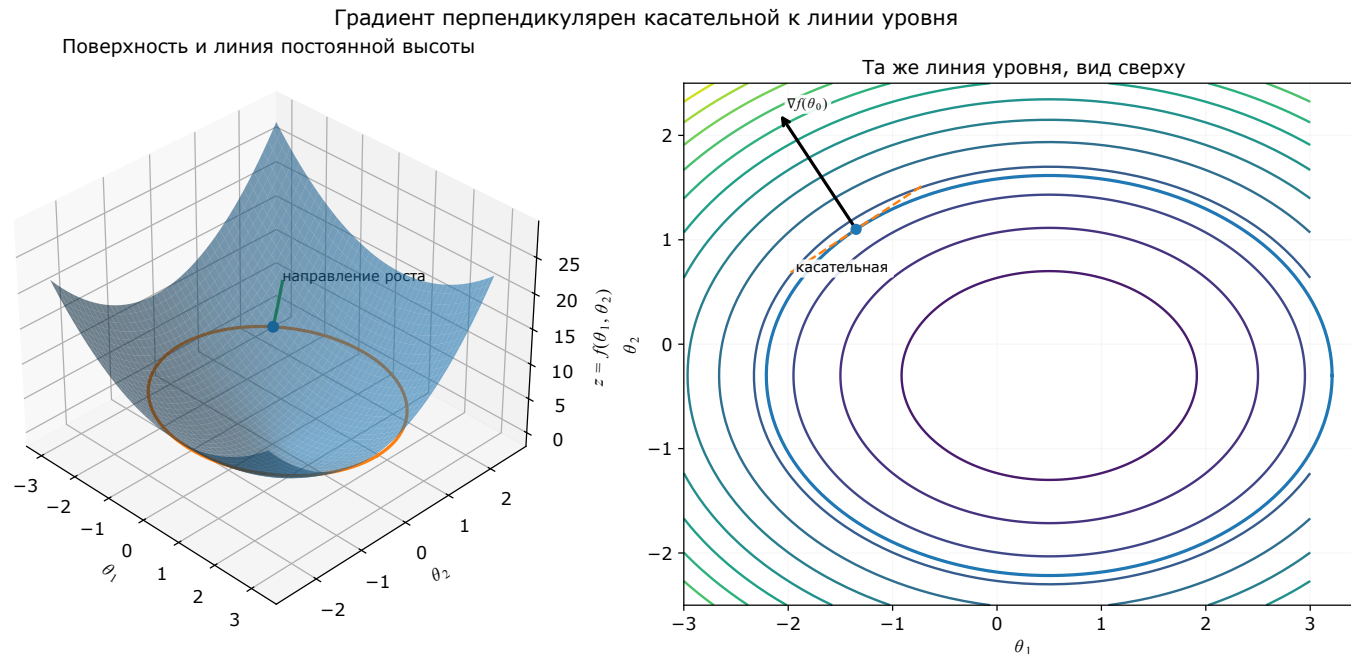
Ненулевой градиент указывает направление наиболее быстрого локального роста.

Антиградиент задаёт направление локального убывания.

Все утверждения локальны: они описывают поведение функции около текущей точки.

Градиент и линии уровня

$$f(\theta_1, \theta_2) = c$$



Вдоль линии уровня значение функции постоянно.

Ненулевой градиент перпендикулярен гладкой линии уровня.

Достаточно малый шаг против градиента уменьшает функцию.

Ставим простейшую задачу минимизации

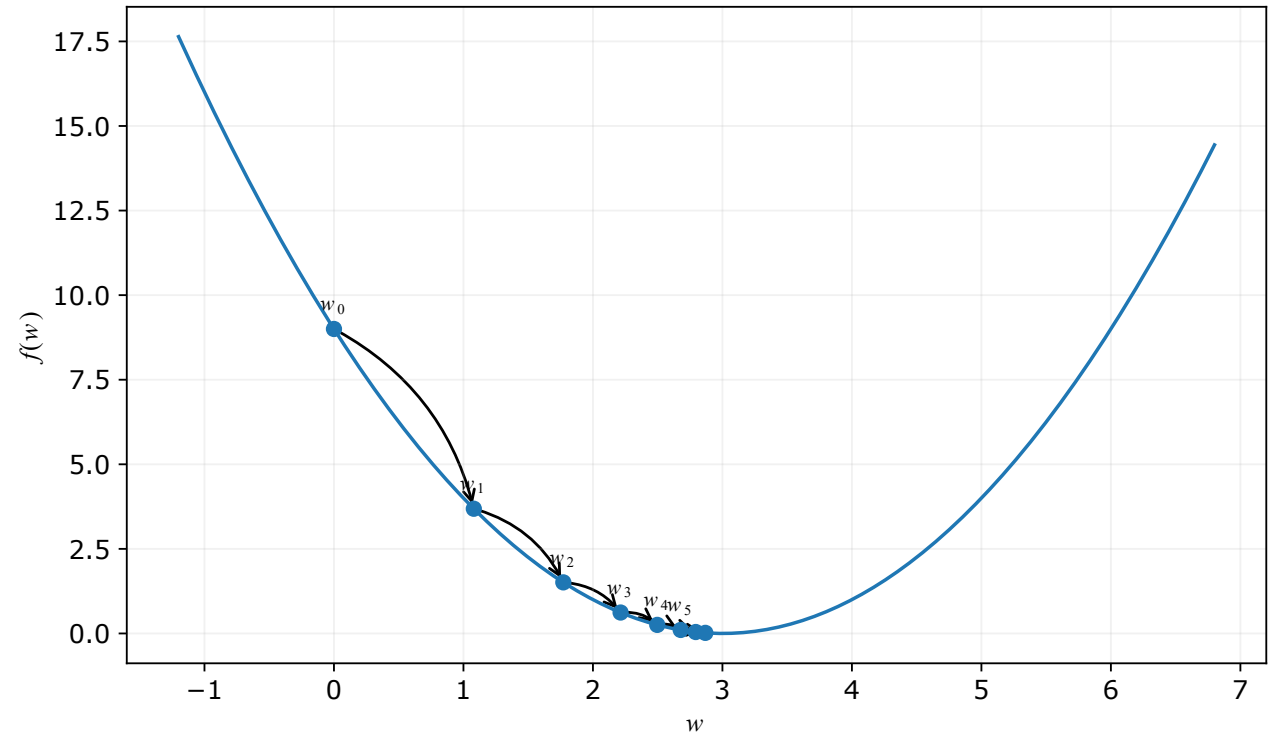
$$\min_w f(w), \quad f(w) = (w - 3)^2$$

$$w_0 = 0, \quad f'(w) = 2(w - 3)$$

$$f'(0) = -6$$

Производная отрицательна: небольшое увеличение w уменьшает функцию.

На каждом шаге локальный наклон вычисляется заново



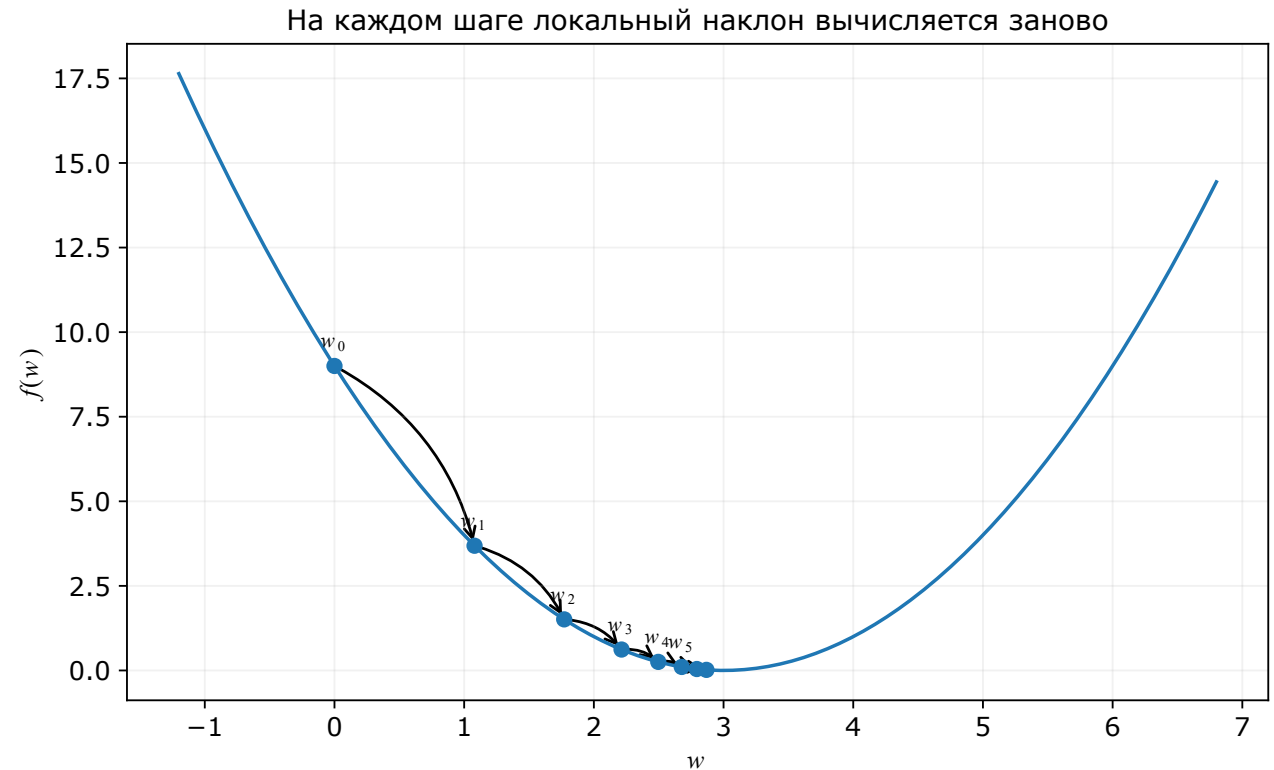
Первый шаг градиентного спуска руками

$$\eta = 0.1$$

$$w_1 = w_0 - \eta f'(w_0) = 0.6$$

$$f'(0.6) = -4.8$$

$$w_2 = 0.6 - 0.1(-4.8) = 1.08$$



После каждого перемещения локальный наклон вычисляется заново.

Градиентный спуск: общий алгоритм

Вычисляем градиент в текущей точке и смещаем параметры в противоположную сторону.

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla R_n(\theta^{(t)})$$

 t

номер итерации

 $\theta^{(t)}$

текущие параметры

 $\eta > 0$

темп обучения

Целевая функция определяет ландшафт; оптимизатор строит по нему последовательность параметров.

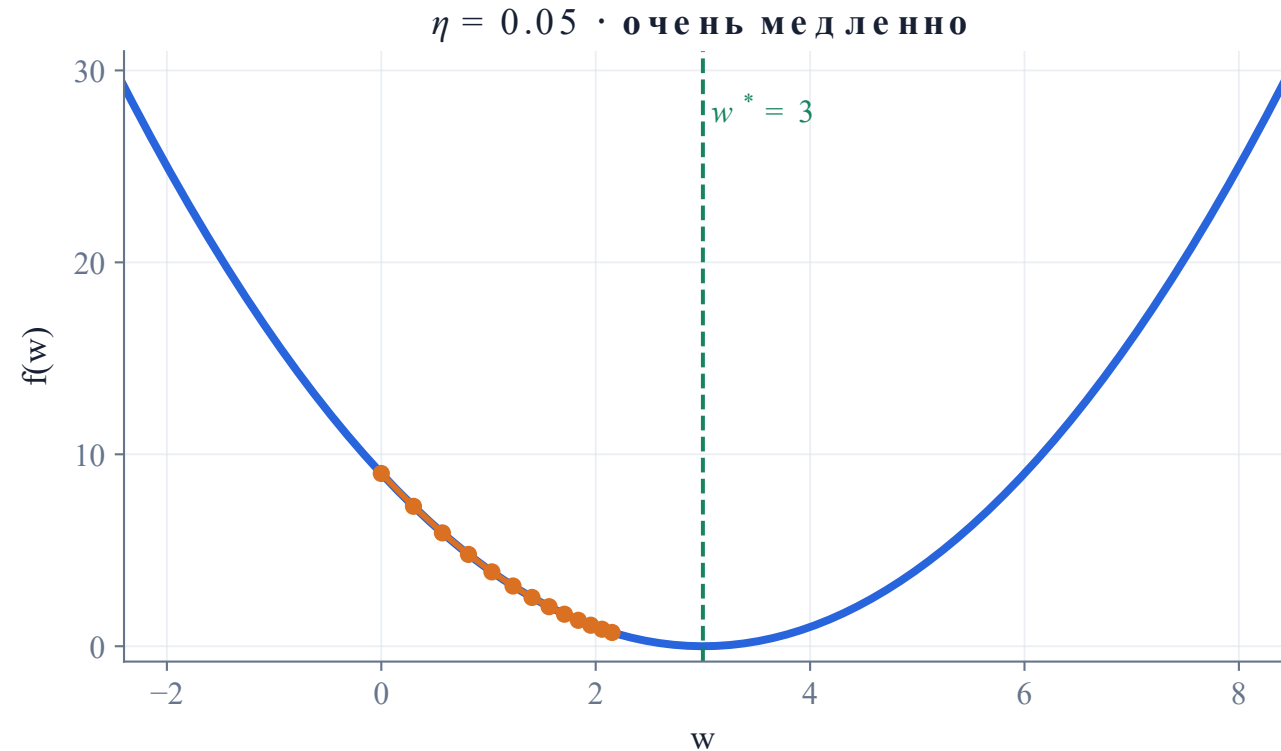
Малый темп обучения

$$f(w) = (w - 3)^2$$

$$\eta = 0.05$$

Движение без перепрыгивания
через минимум.

Очень медленное приближение.



Значение η относится к этой параболе; в другой задаче масштаб темпа обучения будет другим.

Два сходящихся режима движения

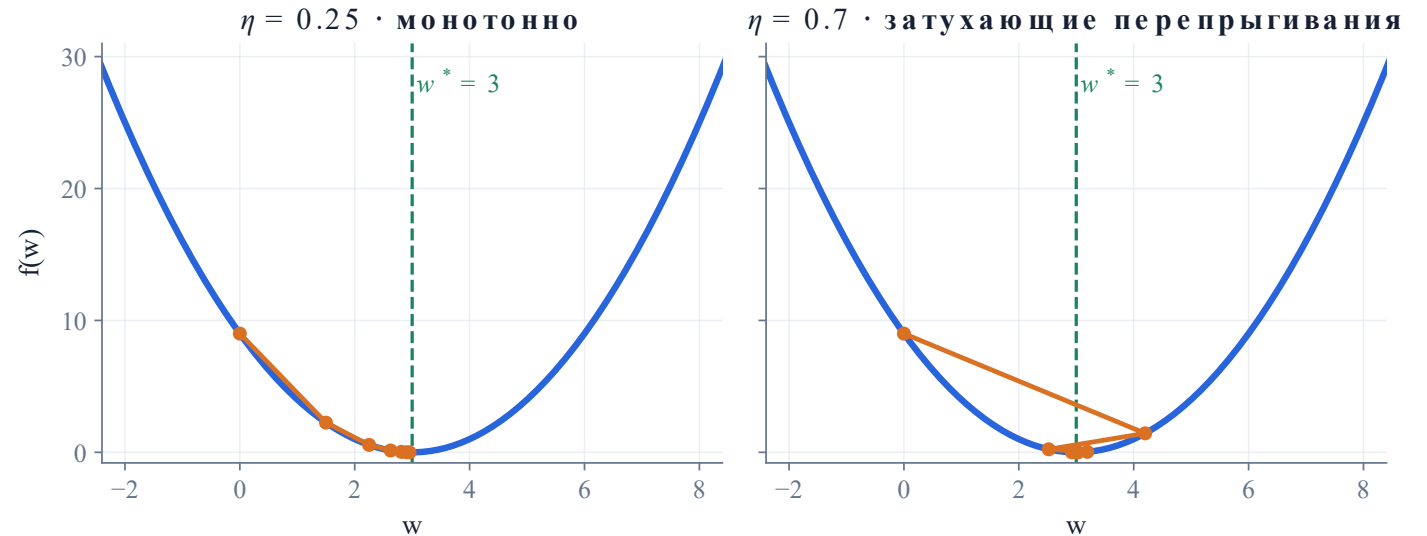
$$f(w) = (w - 3)^2$$

$$\eta = 0.25$$

Быстрое монотонное приближение.

$$\eta = 0.7$$

Сходящиеся колебания.



Показанные значения относятся к этой параболе; в другой задаче масштаб темпа обучения будет другим.

Вечное перепрыгивание и расходимость

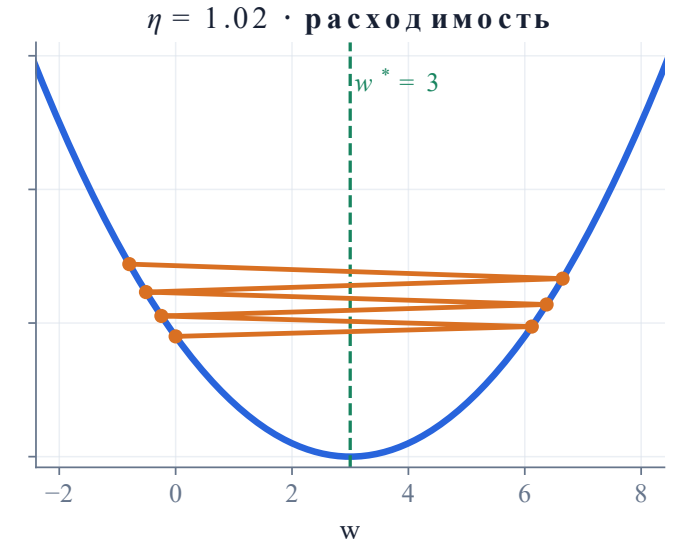
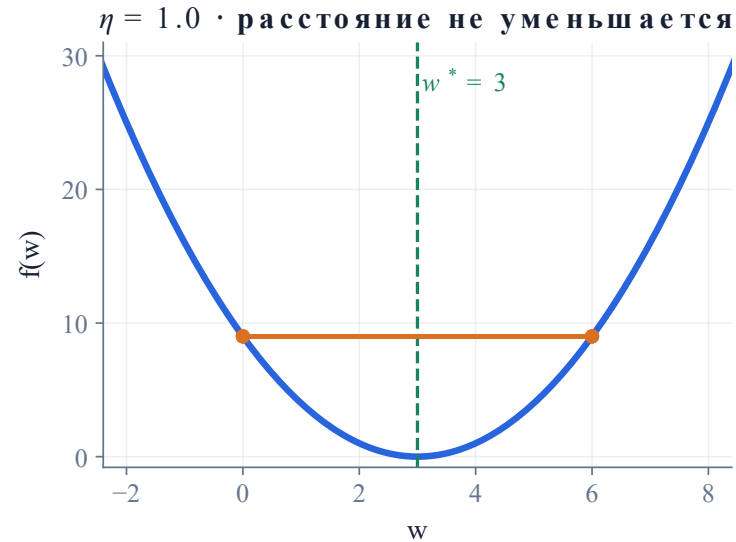
$$f(w) = (w - 3)^2$$

$$\eta = 1$$

Расстояние до минимума не уменьшается.

$$\eta = 1.02$$

Расстояние растёт, итерации расходятся.



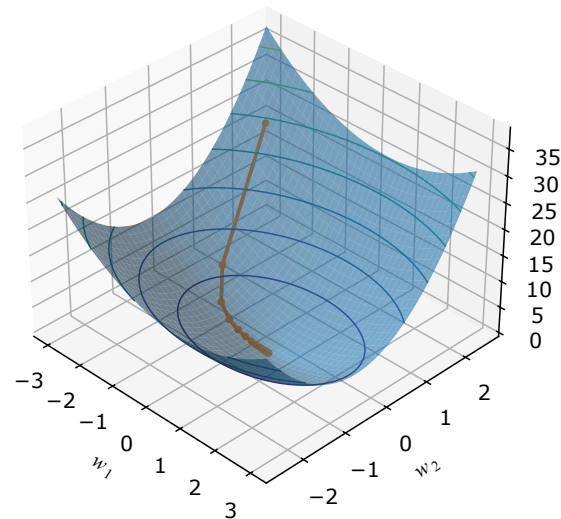
Показанные значения относятся к этой параболе; в другой задаче масштаб темпа обучения будет другим.

Поверхность и карта линий уровня

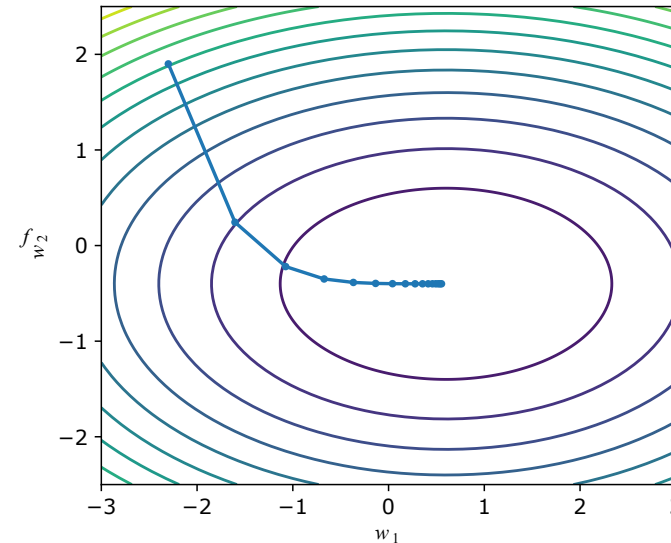
$$f(w_1, w_2) = (w_1 - 0.6)^2 + 3(w_2 + 0.4)^2$$

$$f(w_1, w_2) = (w_1 - 0.6)^2 + 3(w_2 + 0.4)^2$$

Траектория на поверхности



Та же траектория на карте линий уровня



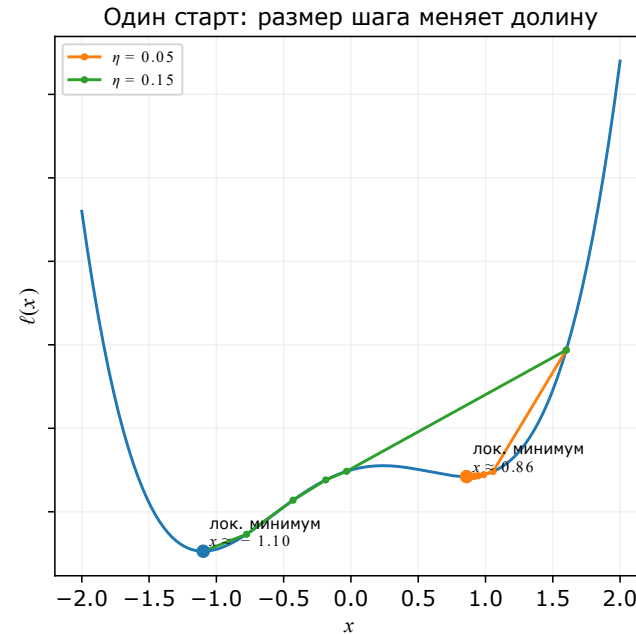
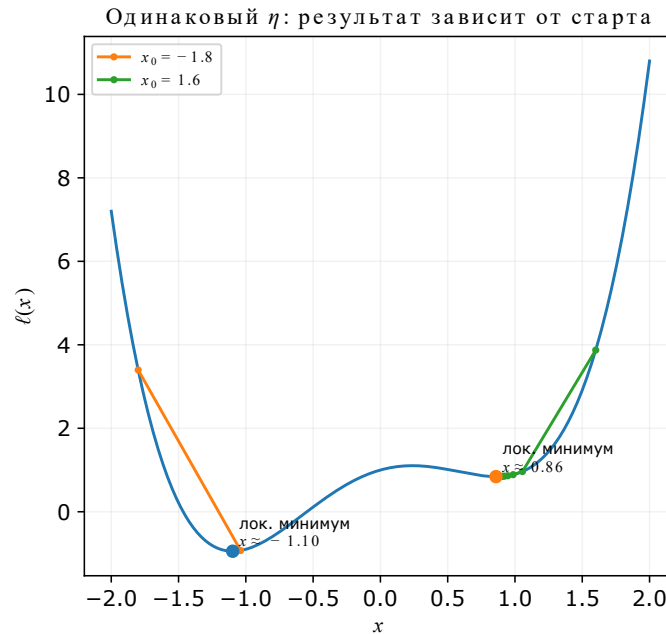
3D-график показывает значение функции третьей координатой.

Карта показывает множества точек с одинаковым значением функции.

На обоих изображениях отмечена одна последовательность параметров.

Невыпуклая функция: локальная информация имеет пределы

$\ell(x) = (x^2 - 1)^2 + 0.9x$: локальные минимумы имеют заметно разную глубину



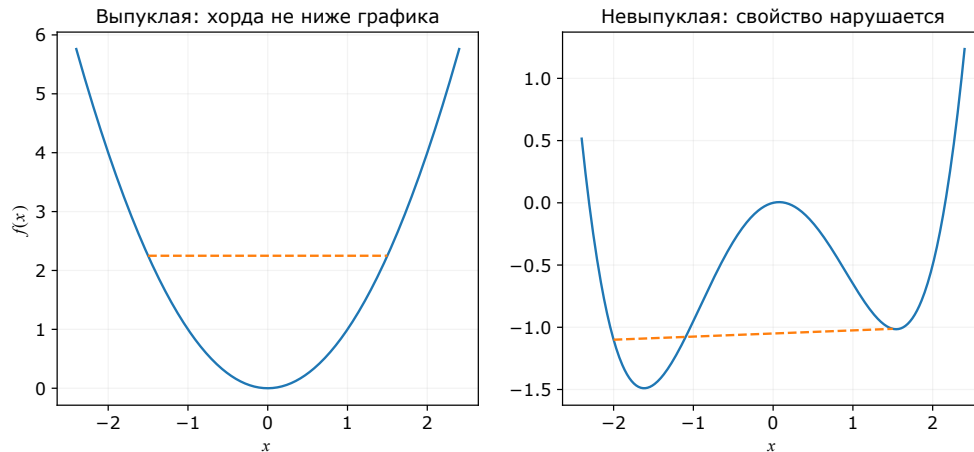
$$\ell(w) = (w^2 - 1)^2 + 0.9w$$

Разные начальные точки могут привести к разным локальным минимумам.

Градиент сообщает только о форме функции в текущей окрестности.

Выпуклость: локальный минимум глобален

Геометрическое определение выпуклости



$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y), \quad 0 \leq \lambda \leq 1$$

У выпуклой функции любой локальный минимум является глобальным.

Глобальный минимум при этом может быть не единственным.

Квадратичная целевая функция линейной регрессии выпукла; при точной мультиколлинеарности множество минимизаторов может содержать несколько точек.

Градиентный спуск итеративный: когда остановиться?

Лимит итераций

$$t = T$$

Малое изменение целевой функции

$$|R(\theta^{(t+1)}) - R(\theta^{(t)})| < \varepsilon$$

Малая норма градиента

$$\|\nabla R(\theta^{(t)})\|_2 < \varepsilon$$

Малое изменение параметров

$$\|\theta^{(t+1)} - \theta^{(t)}\|_2 < \varepsilon$$

Численный критерий остановки не гарантирует ни глобального минимума, ни хорошего качества на новых данных.

Оптимизация на train и качество на новых данных

ОБУЧЕНИЕ

$$\hat{\theta} \in \arg \min_{\theta} R_{\text{train}}(\theta)$$

ОЦЕНИВАНИЕ

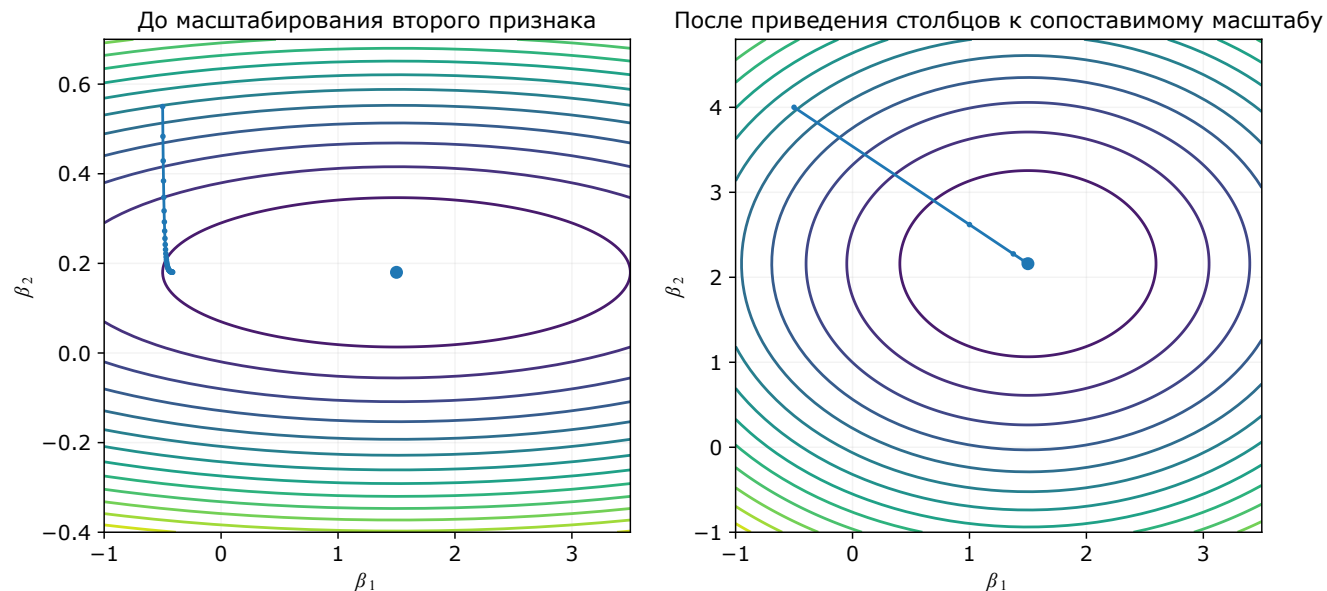
$$M_{\text{val/test}} = M(y, \hat{y}(\hat{\theta}))$$

$$R_{\text{train}} \downarrow \not\Rightarrow M_{\text{val/test}} \text{ улучшается}$$

Оптимизатор работает с обучающей целевой функцией; обобщающую способность проверяют отдельно.

Масштаб признаков меняет геометрию MSE

Масштаб столбцов X меняет форму MSE в пространстве коэффициентов



Разные масштабы признаков дают вытянутые линии уровня.

Один темп обучения трудно подобрать для крутого и пологого направлений.

$$x'_j = \frac{x_j - \mu_j}{s_j}$$

Стандартизация делает масштабы координат сопоставимее.

Параметры преобразования оценивают на train. Масштабирование не добавляет информацию и не устраняет мультиколлинеарность.

Полный градиент: используем всю выборку

$$R_n(\theta) = \frac{1}{n} \sum_{i=1}^n L_i(\theta)$$

$$\nabla R_n(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla L_i(\theta)$$

Все n объектов участвуют перед обновлением.

Получаем точный градиент обучающей целевой функции.

На большой выборке один шаг может быть дорогим.

Mini-batch и SGD: дешевле шаг, шумнее направление

Mini-batch B_t — небольшое подмножество объектов для текущего шага.

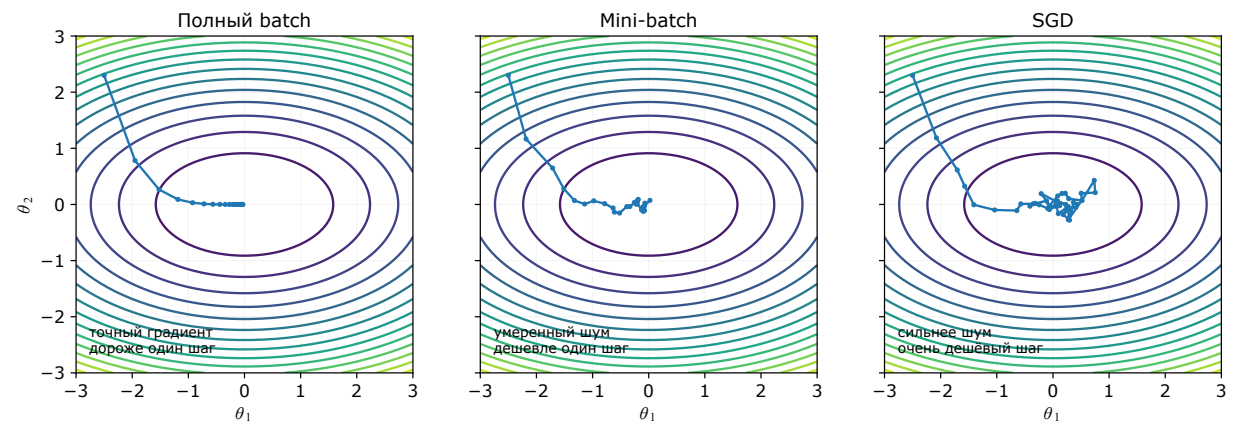
$$g_t = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla L_i(\theta^{(t)})$$

$$\theta^{(t+1)} = \theta^{(t)} - \eta g_t$$

full batch: $|B_t| = n$

SGD: $|B_t| = 1$

Цена одного шага и точность направления: полный градиент, mini-batch и SGD



Модель и функция потерь не меняются; меняется способ оценивания направления шага оптимизатора.

Та же целевая функция МНК, теперь для градиентного спуска

ПРОГНОЗЫ

$$X\beta$$

ОСТАТКИ

$$r(\beta) = y - X\beta$$

ЦЕЛЕВАЯ ФУНКЦИЯ

$$R(\beta) = \frac{1}{n} \|y - X\beta\|_2^2$$

$$R(\beta) = \frac{1}{n} \sum_{i=1}^n r_i(\beta)^2$$

Это та же задача МНК, которую в начале лекции решал `lstsq`.

Один объект: вклад в производную

$$r_i(\beta) = y_i - \sum_{k=1}^q x_{ik} \beta_k$$

$$\frac{\partial r_i}{\partial \beta_j} = -x_{ij}$$



$$\frac{\partial r_i^2}{\partial \beta_j} = 2r_i \frac{\partial r_i}{\partial \beta_j} = -2x_{ij}r_i$$

Вклад объекта — значение признака, умноженное на текущий остаток.

Все координаты собираются в $X^\top r$

$$\frac{\partial R}{\partial \beta_j} = -\frac{2}{n} \sum_{i=1}^n x_{ij} r_i$$

$$X^\top r = \begin{pmatrix} \sum_i x_{i1} r_i \\ \vdots \\ \sum_i x_{iq} r_i \end{pmatrix}$$

$$\nabla R(\beta) = -\frac{2}{n} X^\top (y - X\beta)$$

$$X^\top \in \mathbb{R}^{q \times n}$$

$$r \in \mathbb{R}^n$$

$$X^\top r \in \mathbb{R}^q$$

Один шаг GD для линейной регрессии

$$\beta^{(t+1)} = \beta^{(t)} + \frac{2\eta}{n} X^{\top} (y - X\beta^{(t)})$$

1

Прогнозы

$$X\beta^{(t)}$$

2

Остатки

$$y - X\beta^{(t)}$$

3

Направление

$$X^{\top} r^{(t)}$$

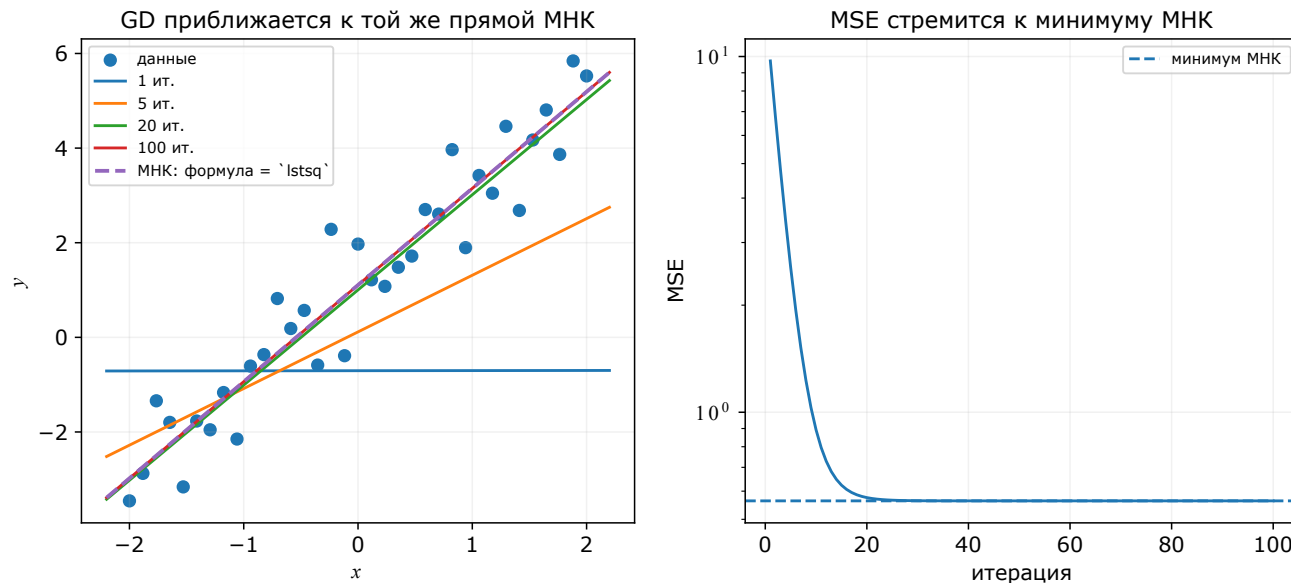
4

Параметры

$$\beta^{(t+1)}$$

Градиент MSE содержит минус, поэтому после подстановки в общее правило обновления появляется плюс.

Та же задача МНК, другой способ решения



$$\min_{\beta} \frac{1}{n} \|y - X\beta\|_2^2$$

`lsqs`

**специализированный численный
решатель least-squares-задачи**

**Градиентный спуск
приближается к тому же минимуму
последовательностью шагов**

Меняется решатель; линейная модель и целевая функция MSE остаются прежними.

Градиент равен нулю → нормальные уравнения

$$\nabla R(\hat{\beta}) = 0$$



$$X^{\top} (y - X \hat{\beta}) = 0$$



$$X^{\top} X \hat{\beta} = X^{\top} y$$

Нормальные уравнения — условие оптимума MSE, а не самостоятельный численный решатель.

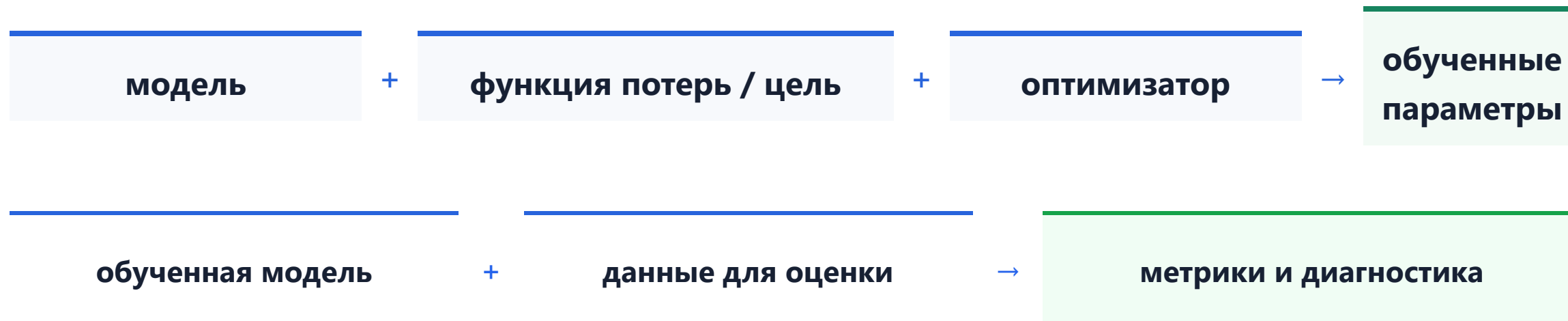
Одна модель — разные целевые функции и способы поиска

Модель	Целевая функция	Способ поиска параметров
линейная	MSE	<code>lstsq</code>
линейная	MSE	полный градиентный спуск
линейная	MSE	mini-batch / SGD
линейная	Huber	градиентный оптимизатор

Меняем оптимизатор → сохраняем задачу выбора параметров.

Меняем функцию потерь → меняем саму задачу оптимизации.

Что скрывается за `fit`



Первые три компонента определяют обучение модели.

Метрики и остатки оценивают уже полученную модель.

Разделяйте четыре вопроса: что моделируем, что минимизируем, как ищем параметры и как оцениваем результат.